

Competitive Programming

Section 51: Invariants In Hard Problems

Competitive Programming Course

2026-07-10

Table of contents

1	Invariants under pressure	3
1.1	What counts as an invariant	3
1.2	Necessity and sufficiency	3
1.3	Parity example	4
1.4	XOR split example	4
1.5	Small-case workflow	4
1.6	Worked residue example	4
1.7	Invariant proof template	5
1.8	How to stress-test an invariant	5
2	Exercises	6
3	Three local invariant workshops	7

1 Invariants under pressure

Hard constructive problems often become simple after finding what never changes: parity, xor, residue class, sorted order, or a conservation law. The workflow is: test small cases, identify the invariant, prove necessity, then show a construction or decision rule for sufficiency.

1.1 What counts as an invariant

An invariant is a property that every allowed operation preserves. If the initial state and target state have different invariant values, the target is impossible.

Common competitive programming invariants:

Operation clue	Likely invariant
add/subtract by 2	parity
add/subtract by k	value modulo k
swap elements	multiset of values
xor operation	xor of selected group
move units between boxes	total sum
reorder under constraints	relative order or inversion parity

1.2 Necessity and sufficiency

Invariant arguments have two halves.

Necessity:

```
Every operation preserves P.  
The start has  $P = A$ .  
Therefore any reachable state must also have  $P = A$ .
```

Sufficiency:

```
If the invariant condition holds, here is a sequence of operations or a direct  
construction that reaches the goal.
```

Many wrong editorials stop after necessity. For red-level problems, the hard part is often proving sufficiency.

1.3 Parity example

If an operation changes a number by 2, then parity never changes. Therefore all numbers can become equal only if they already have the same parity.

That condition is also sufficient: choose any target with that parity, and each number can move to it by repeated +2 or -2 operations.

1.4 XOR split example

For an array, define prefix xor $px[i] = a[1] \text{ xor } \dots \text{ xor } a[i]$. The xor of the left part before split i is $px[i]$. The xor of the right part is:

```
total_xor xor px[i]
```

So the split is balanced iff:

```
px[i] == total_xor xor px[i]
```

Equivalently, $total_xor == 0$. If total xor is not zero, no split works; if it is zero, every split with equal prefix and suffix xor can be counted directly.

1.5 Small-case workflow

When stuck:

1. Generate all states for $n \leq 5$.
2. Group reachable states by parity, sum, xor, or residue.
3. Look for a property shared by every state in a component.
4. Try to construct moves between any two states with the same property.

The local exercises are intentionally small, but this workflow is exactly what scales to Codeforces constructive problems.

1.6 Worked residue example

Operation:

```
choose one number and add or subtract k
```

Array:

```
[5, 11, 17], k = 6
```

All values are $5 \bmod 6$, so they can all become equal. For example, make them all 11:

```
5 + 6 = 11
11 stays
17 - 6 = 11
```

Array:

```
[5, 10, 17], k = 6
```

Residues are:

```
5, 4, 5
```

The second number can never have residue 5, so equality is impossible.

1.7 Invariant proof template

Use this exact structure in editorials:

```
Invariant: value modulo k.
Preservation: adding/subtracting k does not change modulo k.
Necessity: if two values have different residues, they can never be equal.
Sufficiency: if all residues match, move every value to one chosen target by
repeated +/- k operations.
```

1.8 How to stress-test an invariant

For a constructive invariant problem, brute force tiny states:

```
states: all arrays of length <= 4 with values 0..5
edges: one allowed operation
components: reachable sets under those operations
```

Then print candidate invariant values for each component. If every component has one invariant value, the invariant may be necessary. If every state with that invariant value is in the same component, it may also be sufficient.

This is not a proof, but it is a powerful way to discover the proof.

2 Exercises

- A. Same Parity: Can all numbers become equal if one operation changes a number by 2?
- B. XOR Balanced Splits: Count split positions where left xor equals right xor.
- C. Same Residue: Can all numbers become equal using operations that change a number by k ? # Contest checklist

3 Three local invariant workshops

Parity. Adding or subtracting two preserves $x \bmod 2$. For 1,5,7,9, all values are odd, so each can move through odd integers toward a common target. For 1,4, no sequence of ± 2 operations can bridge the residue classes. The proof has two parts: preservation makes mixed parity impossible; repeated steps of two make equal-parity values reachable at a common sufficiently large value.

XOR split. Let T be XOR of the whole array and P XOR of the prefix before a cut. The suffix XOR is $T \text{ xor } P$; equality holds exactly when $P == T \text{ xor } P$, equivalently $T == 0$. Thus when total XOR is zero every cut is valid, otherwise none is. Test $n=1$, all zero, and a nonzero total; do not scan each suffix independently.

Residue modulo k . $\pm k$ preserves $x \bmod k$. Compare normalized residues, especially if a future variant permits negative values. Mixed residues are impossible; equal residues are sufficient by repeatedly moving each value by k .

C++/Python and workflow. Use `long long` for values and XOR in C++; Python integers are exact. Brute-force small operation graphs to discover a candidate invariant, then prove preservation and sufficiency separately. Run `CP_TARGET=solution python3 sections/51_invariants_in_hard_problems/check.py`.

Exercises: explain why a preserved quantity alone is not always sufficient; find the parity classes of $[-3,2,7]$; and verify the XOR identity on $[1,2,3,0]$. Answers: reachability may have another obstruction; residues are 1,0,1; total XOR is zero, so every internal cut works.

State the invariant, choose the model, prove the operation, then code the smallest reliable version.